

pyharm: Analysis and Plotting for GRMHD

Cora Prather ¹ 

¹ Black Hole Initiative, Harvard University  Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#))

Summary

pyharm is a scalable python library for plotting and analyzing some general-relativistic magnetohydrodynamic (GRMHD) simulations, especially those simulating matter around black holes. It handles reading a number of different simulation snapshot formats, plotting basic and derived fluid properties in different spacetimes and coordinate systems, and computing arbitrary reductions across large datasets of fluid snapshots.

Statement of Need

As large, complex 3D datasets, GRMHD simulation snapshots require sophisticated post-processing and plotting code in order to correctly interpret their data values. Writing new plotting scripts from scratch takes time and effort, not to mention being bug-prone due to the overall complexity. A framework which pre-defines variables and coordinates consistently saves time in understanding these datasets, and improves the quality of results by allowing fixes to be preserved in all future calculations.

State of the Field

Many plotting libraries and utilities exist for visualizing and analyzing eulerian hydrodynamics simulations at scale, e.g. ([Turk et al., 2010](#)), ([Bethel et al., 2012](#)), ([Ahrens et al., 2005](#)). However, GRMHD in particular presents two unique challenges that make these tools difficult to use or wholly insufficient:

1. An analytic transformation is often made from a “natural” coordinate system (r, θ, ϕ in the spacetime) into a separate coordinate system where the simulation grid is regular. Imaging simulations requires performing the reverse transformation.
2. Fluid quantities in GRMHD do not generally match the quantities in Newtonian MHD, and generally require the local metric information, and potentially frame transformations, to compute.

Plotting scripts exist which natively support GRMHD grids and quantities, but are generally either private, or specialized to a particular code or style of GRMHD ([Bozzola, 2021](#)). pyharm attempts to support as many codes as possible.

Code Design

The central class in pyharm is the `FluidState`, representing a snapshot of a GRMHD simulation at some point in time (either generated, memory-backed, or file-backed, generally the latter). Data retrieval and manipulation are both done by means of the `__getitem__` function, allowing users to treat the object as a dictionary containing any desired derived variable. When a key is retrieved, e.g. `dump['beta']` for the plasma β parameter (ratio of gas pressure to

37 magnetic pressure), it is computed on the fly, returned, and cached in memory for the next
38 use. Quantities can be defined in terms of one another recursively, and thus the code avoids
39 repeating common calculations.

40 While pyharm is designed first as a python library for user scripts, it has gained a number of
41 useful command-line utilities, two of which are easily extensible and broadly useful as a way of
42 accessing pyharm's functionality.

43 pyharm movie handles slice and average plots per-snapshot across a simulation (including
44 optionally encoding the results using ffmpeg). Large jobs consisting of many frames can be
45 parallelized across multiple cores or nodes with Python's built-in multiprocessing or with
46 mpi4py.futures (Rogowski et al., 2023). New movies can be defined solely by adding a
47 function in pyharm/plots/figures.py which creates one frame, with the parallelism features
48 shared between all functions.

49 pyharm analysis handles reductions over snapshots from an entire simulation (slices, time-
50 averages, flow rates, etc.). Results are written to a single HDF5 file collecting from all
51 processes/ranks, in a standard layout readable by another command-line script, pyharm plot-
52 result.

53 In addition to the main scripts, pyharm provides a number of small utilities for checking
54 simulation logs for progress and errors, checking validity of simulation snapshots, or checking
55 snapshots against one another.

56 Implementation Notes

57 pyharm is written in pure Python using numpy, scipy, and matplotlib. As it is implemented
58 purely with array operations, not loops, the code shares numpy and scipy's good efficiency
59 on single CPU cores. Parallelizing operations on a single dump is not a goal of the code –
60 usually, compute-intensive operations are conducted across many dump files in a way which
61 is substantially or wholly embarrassingly parallel. Likewise, GPUs are not a priority as many
62 operations are already significantly bottlenecked by file read speed. Instead, pyharm achieves
63 good speed mostly by avoiding unnecessary read and compute operations.

64 As its most important optimization, pyharm supports slicing of HDF5-based dump files without
65 first reading the entire grid into memory. Thus if only, e.g., a slice at $\phi = 0$ is required, pyharm
66 supports a slicing operation `sliced_dump = dump[:, :, 0]` before any reads are performed, and
67 will read only the given slice from the data file. Most operations are supported on sliced files,
68 including the calculation of derived variables, reductions, and plotting.

69 In addition, the (sliced) arrays corresponding to each key are natively cached by the pyharm
70 data objects: file readers can cache the variables retrieved by a FluidState object, and
71 FluidState objects themselves can cache the results of computations to avoid repeating them.
72 Caching can be memory-intensive, but when combined with slicing it can lead to dramatic
73 improvements in runtime of complex analyses and reductions which share basic components.

74 With these optimizations, pyharm can perform simple movies at great speed: XX frames/s on
75 64 cores of a single CPU node on the Harvard Cannon cluster, XX frames/s on ALCF Crux.
76 More complicated analyses, for example `example_analysis` with computations of XX, YY, ZZ,
77 are performed at XX dumps/s on Cannon, YY dumps/s on Crux.

78 Tests

79 As primarily a plotting framework, most bugs or undesired behaviors in pyharm cannot be
80 easily tested in an automated way. However, calculations of physical quantities are tested
81 against expectations, such as an example calculation of 4-vector quantities [here](#). “Golden

file” regression tests of the calculation of basic reductions are also available, over an array of different file formats.

Research projects using pyharm

pyharm has been used in obtaining the results of many published papers, especially from the GRMHD code KHARMA (Prather, 2025). This includes collaboration papers from the EHT such as (Event Horizon Telescope Collaboration et al., 2019) and (Event Horizon Telescope Collaboration et al., 2022), and independent papers using KHARMA, e.g. most recently (?), (Bukowiecka et al., 2026) and (Stanway et al., 2026).

Acknowledgements

Work supported by the Black Hole Initiative at Harvard University, which is funded in part by the Gordon and Betty Moore Foundation (grant #13526). It was also made possible through the support of a grant from the John Templeton Foundation (grant #63445). The opinions expressed in this publication are those of the author(s) and do not necessarily reflect the views of these Foundations.

An award of computer time was provided by the U.S. Department of Energy’s (DOE) Innovative and Novel Computational Impact on Theory and Experiment (INCITE) Program. This research used supporting resources at the Argonne and the Oak Ridge Leadership Computing Facilities. The Argonne Leadership Computing Facility at Argonne National Laboratory is supported by the Office of Science of the U.S. DOE under Contract No. DE-AC02-06CH11357. The Oak Ridge Leadership Computing Facility at the Oak Ridge National Laboratory is supported by the Office of Science of the U.S. DOE under Contract No. DE-AC05-00OR22725.

This work used Delta CPU at the National Center for Supercomputing Applications through allocation PHY250339 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296.

AI usage disclosure

No AI tools were used in developing pyharm or this paper.

References

- Ahrens, J., Geveci, B., & Law, C. (2005). ParaView: An End-User Tool for Large-Data Visualization. In *Visualization Handbook* (pp. 717–731). Elsevier. <https://doi.org/10.1016/B978-012387582-2/50038-1>
- Bethel, E. W., Childs, H., & Hansen, C. (Eds.). (2012). *High Performance Visualization: Enabling Extreme-Scale Scientific Insight* (0th ed.). Chapman and Hall/CRC. <https://doi.org/10.1201/b12985>
- Bozzola, G. (2021). Kuibit: Analyzing Einstein Toolkit simulations with Python. *Journal of Open Source Software*, 6(60), 3099. <https://doi.org/10.21105/joss.03099>
- Bukowiecka, N., Ricarte, A., Kocherlakota, P., & Prather, C. (2026). Observational distinguishability of the Kerr and Kerr-Hayward metrics to EHT. *Physical Review D*, 114(2), 023008. <https://doi.org/10.1103/rfrx-tfz8>
- Event Horizon Telescope Collaboration, Akiyama, K., Alberdi, A., Alef, W., Algaba, J. C., Anantua, R., Asada, K., Azulay, R., Bach, U., Baczko, A.-K., Ball, D., Baloković, M.,

- 123 Barrett, J., Bauböck, M., Benson, B. A., Bintley, D., Blackburn, L., Blundell, R., Bouman,
124 K. L., ... White, C. (2022). First Sagittarius A* Event Horizon Telescope Results. V.
125 Testing Astrophysical Models of the Galactic Center Black Hole. *The Astrophysical Journal*
126 *Letters*, 930(2), L16. <https://doi.org/10.3847/2041-8213/ac6672>
- 127 Event Horizon Telescope Collaboration, Akiyama, K., Alberdi, A., Alef, W., Asada, K., Azulay,
128 R., Baczko, A.-K., Ball, D., Baloković, M., Barrett, J., Bintley, D., Blackburn, L., Boland,
129 W., Bouman, K. L., Bower, G. C., Bremer, M., Brinkerink, C. D., Brissenden, R., Britzen,
130 S., ... Zhang, S. (2019). First M87 Event Horizon Telescope Results. V. Physical Origin of
131 the Asymmetric Ring. *The Astrophysical Journal*, 875, L5. <https://doi.org/10.3847/2041-8213/ab0f43>
- 132
- 133 Prather, C. (2025). KHARMA: Flexible, Portable Performance for GRMHD. In C. Bambi,
134 Y. Mizuno, S. Shashank, & F. Yuan (Eds.), *New Frontiers in GRMHD Simulations* (pp.
135 167–201). Springer Nature Singapore. https://doi.org/10.1007/978-981-97-8522-3_5
- 136 Rogowski, M., Aseeri, S., Keyes, D., & Dalcin, L. (2023). Mpi4py.futures: MPI-Based
137 Asynchronous Task Execution for Python. *IEEE Transactions on Parallel and Distributed*
138 *Systems*, 34(2), 611–622. <https://doi.org/10.1109/TPDS.2022.3225481>
- 139 Stanway, J. S., Prather, C., Ward-Thompson, D., Walton, T. J., Patterson, B., & Cho, H.
140 (2026). *Supermassive Black Holes: Modelling strongly and weakly magnetised misaligned*
141 *accretion disks* (No. arXiv:2605.27334). arXiv. <https://doi.org/10.48550/arXiv.2605.27334>
- 142
- 143 Turk, M. J., Smith, B. D., Oishi, J. S., Skory, S., Skillman, S. W., Abel, T., & Norman,
144 M. L. (2010). Yt: A MULTI-CODE ANALYSIS TOOLKIT FOR ASTROPHYSICAL
145 SIMULATION DATA. *The Astrophysical Journal Supplement Series*, 192(1), 9. <https://doi.org/10.1088/0067-0049/192/1/9>
- 146